

Masterplan: Transformer Inference Accelerator

A solo, from-scratch chip design project - target: Etched RTL Intern (ASIC), San Jose

This is an independently designed transformer inference accelerator - its own ISA, its own precision plan, its own memory hierarchy and dataflow, chosen from first principles for what a transformer actually needs (dense matmul, attention, softmax, KV cache), not adapted from a sparse-SNN design.

The bar for scope and rigor is set by two things you've already touched: the IITM Dual-Sparse SNN Accelerator (full-chip ISA, DMA/AXI, double-buffered SRAM, scheduler, integration Makefile, full verification stack) and the COMPASS-CNN HPCA 2027 submission (power/roofline/energy analysis rigor). Those are used here strictly as a *complexity and correctness reference* - proof that a single accelerator project can span ISA to synthesis and be fully verified. Every actual design decision below - precision, dataflow, memory sizing, ISA fields - is worked out fresh for a transformer, because a transformer's dense, regular, attention-heavy compute profile is a genuinely different design problem from sparse spiking CNN inference.

1. Why This Project, At This Scope

Etched's JD asks for: microarchitecture design, SystemVerilog RTL, the full cycle from microarch discussion through synthesis and timing feedback, and the ability to learn transformer/LLM concepts quickly. They explicitly say prior ML/AI hardware experience is *not required*, but familiarity with modern LLM architectures and numerical representations is a plus. Their product (Sohu) is a transformer-only ASIC - the entire company thesis is that baking the transformer's structure into silicon beats a general-purpose GPU for inference.

Your existing RTL and verification-flow experience is strong and already proven. What isn't yet proven is that you can independently reason about a transformer's dataflow and numerics - matmul-heavy compute, attention's very different memory access pattern, softmax as a hardware primitive, and why decode-time inference is bandwidth-bound in a way CNN/SNN inference generally isn't. This project is built to demonstrate exactly that, solo, end to end.

Scope bar (complexity/correctness reference, not a template to copy)

- A full custom ISA, opcode-dispatched from a modeled instruction stream in DRAM.
- A double-buffered, multi-bank on-chip SRAM subsystem sized from first-principles arithmetic for transformer tensors (not convolution tiles).
- AXI-style DMA read/write engines moving weights, activations, and KV-cache data.
- A compute core built specifically for transformer math: a dense systolic matmul array plus a dedicated attention/softmax datapath - two different execution units, not one reused SNN PE.
- A scheduler/controller orchestrating multi-instruction execution waves across these units.
- A four-piece verification stack: golden Python reference model, instruction generator, software instruction-level simulator, and a SystemVerilog testbench doing a bit-exact DRAM-output comparison.
- A synthesis run (open-source flow or institute EDA tools) with area/power numbers and a roofline analysis, written up with the same quantitative rigor as a paper draft.

2. Reading List - Read Thoroughly, In This Order

Each entry states why it matters for *this specific design*, not just in general. You need to be able to defend design choices against these papers in an interview, not just cite them.

2.1 Foundations - the model itself

Paper	Why it's non-negotiable
Vaswani et al., "Attention Is All You Need" (2017)	The architecture you're building hardware for. Be able to derive Q/K/V projections, scaled dot-product attention, multi-head split/concat, and the FFN block from memory, and map each op to a hardware primitive before designing the compute units.
Radford et al., GPT-2 (2019)	The concrete decoder-only variant to target: causal masking, autoregressive decode with a growing KV cache. Etched is inference- and decode-latency-focused, so understanding decode vs. prefill compute/memory balance matters directly.
Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need" (Multi-Query Attention, 2019)	Explains why KV-cache bandwidth dominates decode cost - this should directly inform your KV-cache memory-hierarchy decisions.
Dao et al., "FlashAttention" (2022) / FlashAttention-2 (2023)	The modern software dataflow for attention that avoids materializing the full attention matrix - decide explicitly whether your attention unit adopts a similar tiled dataflow, and justify it either way in the writeup.

2.2 Numerics and quantization - decide these for a transformer, not by reusing SNN precision choices

Paper	Why it matters here
Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference" (2018)	Baseline affine INT8 quantization (scale + zero-point) for weights and activations - the reference point for your matmul engine's numerics.
Dettmers et al., "LLM.int8()" (2022)	Transformer activations develop large outliers at scale, which breaks naive per-tensor INT8. Decide and document how your design would handle this (per-channel scaling, mixed precision, or an explicit documented limitation).
Micikevicius et al., "FP8 Formats for Deep Learning" (2022)	Modern LLM inference chips increasingly use FP8 (E4M3/E5M2) rather than INT8. Since Etched's JD flags numerical representations explicitly, you should at minimum design your PE's accumulator and datapath so FP8 is a documented, reasoned-about alternative, even if you implement INT8 first.

2.3 Accelerator architecture canon

Paper	Why it matters here
Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit" (ISCA 2017)	The reference systolic-array matmul accelerator. Study its weight-stationary dataflow and systolic timing in detail - this is the architectural template for your matmul engine.

Paper	Why it matters here
Chen et al., "Eyeriss" (ISCA 2016 / JSSC 2017)	The standard dataflow taxonomy (weight-/output-/row-stationary). You need to be able to name and justify your own dataflow choice against these alternatives from first principles.
Genc et al., "Gemmini" (DAC 2021)	An open, well-documented systolic-array generator - useful as an architecture and RTL-structure reference independent of any SNN work.
Williams, Waterman, Patterson, "Roofline" (CACM 2009)	Read the original source before producing your own roofline plot, so arithmetic intensity and the compute/memory-bound ridge point are used correctly, not just copied as a chart format.

2.4 Transformer/LLM-specific accelerators (closest public comparisons to Etched)

Paper	Why it matters here
Ham et al., "A^3" (HPCA 2020)	One of the first dedicated attention accelerators - a reference dataflow for the attention unit.
Ham et al., "ELSA" (ISCA 2021)	Hardware-software co-designed approximate attention - a useful comparison point when you explain what you deliberately left out of scope.
Qin et al., "FACT" (ISCA 2023)	A recent full-transformer accelerator covering both attention and FFN end to end - closest in shape to what you are building; study how it structures the whole chip, not just one unit.
Zeng et al., "FlightLLM" (FPGA 2024)	An LLM-inference-specific accelerator with explicit KV-cache handling and decode/prefill distinction - the closest public analogue to Etched's actual problem. Study its memory hierarchy and dataflow closely.
Etched, public material at etched.com and any conference talks/blog posts on Sohu	Not a paper, but read everything public. Be ready to say specifically how your design choices relate to, or diverge from, what's publicly known about Sohu, and why.

2.5 Complexity/rigor references only

- IITM Dual-Sparse SNN Accelerator BTP report - read the verification chapter (golden reference model -> instruction generator -> software instruction simulator -> RTL testbench -> byte-compare) as a template for how thorough your own verification writeup should be. Do not port its ISA fields, SRAM sizing formulas, or PE microarchitecture directly - those are CNN/SNN-specific and would be the wrong numbers for a transformer.
- COMPASS-CNN HPCA 2027 submission - read as a template for how a chip-design report presents quantitative results (area/power breakdowns, roofline plots, iso-accuracy energy comparisons). Again: reference for rigor and presentation style, not for architecture content.

3. Target Architecture - Transformer-Native Design

Every choice below is derived from what a transformer inference workload actually needs, worked out independently rather than adapted from a CNN/SNN accelerator.

3.1 Top-level dataflow

DRAM (quantized weights, input activations, growing KV cache) -> AXI-style Read DMA -> double-buffered on-chip SRAM (activation, weight, KV-cache, output banks) -> Instruction Dispatcher -> {Matmul Engine | Attention/Softmax Unit | LayerNorm/Elementwise Unit}, each a distinct execution unit sized for its own operation profile -> Output SRAM -> Write DMA -> DRAM.

3.2 Scope: which transformer operations get dedicated hardware

Op	In scope?	Rationale
QKV projection (matmul)	Yes - matmul engine	The dominant FLOP consumer in any transformer layer; must be fast, dense, and pipelined.
Scaled dot-product attention (QK ^T , softmax, x V)	Yes - dedicated unit	Attention has a fundamentally different access pattern from a plain matmul (score matrix, row-wise softmax, second matmul) and deserves its own datapath rather than being forced through the matmul engine.
Softmax	Yes - simplified (max-subtract, LUT-based exp, reciprocal)	A real hardware design problem in its own right: numerical stability, LUT sizing, and normalization latency are worth reasoning about carefully and documenting.
FFN matmul	Yes - reuses the matmul engine as a generic tiled GEMM unit	No new hardware needed if the matmul engine is operand-agnostic; a good efficiency point to highlight in the writeup.
LayerNorm	Yes - mean/variance/normalize pipeline	Small, bounded scope, but a real accelerator still needs it; shows you don't skip the non-matmul plumbing.
KV-cache management	Yes - dedicated double-buffered SRAM bank plus append-only write logic keyed on sequence position	The single most transformer-inference-specific structural decision in the whole design; this is where decode-time bandwidth pressure actually shows up in hardware.
Multiple layers / full model	No - one configurable layer	Multi-layer tiling is a scaling problem, not an architecture problem; one fully verified layer proves the design.
Training / backward pass	No	Out of scope by definition; inference only, matching Etched's own product focus.

3.3 Precision plan (decided for a transformer workload)

Transformers are far more sensitive to naive low-precision quantization than CNNs, mainly because of activation outliers that grow with model scale (see Dettmers et al., 2.2). The precision plan below is chosen for that reason, not carried over from any prior sparse-SNN design:

- Weights: INT8, per-channel (per-output-feature) scale factors rather than one scale per tensor - cheap in hardware (one extra multiply per output channel) and substantially better accuracy than per-tensor scaling.

- Activations: INT8 with per-tensor dynamic scaling computed at runtime for matmul inputs; explicitly document the outlier problem and, as a stretch goal, support a small set of higher-precision (INT16) lanes for outlier channels, similar in spirit to LLM.int8() but adapted to a fixed-function datapath rather than a software runtime.
- Accumulation: INT32 in the matmul engine to avoid overflow across long reduction dimensions (model dimension `d_model` can be several hundred to a few thousand).
- Attention scores and softmax: kept at higher internal precision (INT16 or a small fixed-point format) before the LUT-based exponential, since softmax is numerically sensitive and this is a well-known failure point in low-precision attention hardware.
- State explicitly in the writeup why FP8 was considered and either adopted or deferred - this is a live, current design decision in real LLM accelerators and interviewers will likely ask about it.

3.4 Draft ISA

128-bit instruction words, opcode-routed - a conventional and defensible instruction-word convention for a tiled dataflow accelerator, chosen on its own merits for this design.

Opcode	Mnemonic	Function
0x0	LOAD_ACT	DMA read of input activations, DRAM -> activation SRAM (double-buffered)
0x1	LOAD_WGT	DMA read of per-channel-quantized weights, DRAM -> weight SRAM
0x2	MATMUL	Dispatch one tile through the systolic matmul engine (used for QKV projection and FFN)
0x3	ATTN	Dispatch one attention tile: $QK^T \rightarrow \text{softmax} \rightarrow x V$ through the attention unit
0x4	LN	LayerNorm pass over a tile resident in SRAM
0x5	KV_APPEND	Append newly computed K/V tile into the KV-cache SRAM bank at the current sequence position
0x6	STORE_OUT	Write DMA: output SRAM -> DRAM

3.5 Memory system

- Independent double-buffered SRAM banks for activations, weights, KV cache, and output, each with its own arbiter enforcing a single invariant: the two requesters on a bank never target the same physical half in the same cycle. This invariant, not any specific bank sizing, is the reusable engineering idea worth carrying forward from prior double-buffered SRAM design experience.
- Size every bank from transformer-specific first-principles arithmetic in the report: activation SRAM from tile width $\times d_{\text{model}} \times \text{precision}$, weight SRAM from tile size $\times d_{\text{model}} \times d_{\text{ff}}$, and critically the KV-cache bank from $\text{max_sequence_length} \times \text{num_heads} \times \text{head_dim} \times 2$ (K and V) $\times \text{precision}$ - this sizing derivation is the single most transformer-specific piece of memory-system design work in the project and should be shown explicitly, not assumed.
- A single AXI-style Read DMA and Write DMA engine is sufficient scope; a from-scratch, transformer-sized memory system does not need to replicate every DMA-path split of a larger multi-person project.

3.6 Compute units

- Matmul engine: an $N \times N$ weight-stationary systolic array (start 8×8 , stretch to 16×16), INT8 operands with per-channel weight scaling, INT32 accumulation. Systolic weight-stationary dataflow is chosen because

transformer matmuls are dense and regular - there is no sparsity to exploit, so a sparse-operand-matching PE design would be the wrong tool here; a classic TPU-style dense systolic array is the correct, deliberate choice, and the writeup should say so explicitly.

- Attention/softmax unit: a separate, smaller datapath - QK^T via the same or a scaled-down systolic block, a row-wise max-subtract and LUT-based exponential, a reciprocal/normalize stage, and a second matmul (attention weights $\times V$). This unit, sized and pipelined specifically for attention's access pattern, is the most original piece of RTL in the project and deserves the most design time.
- LayerNorm/elementwise unit: a small streaming mean/variance/normalize pipeline, kept intentionally simple so it doesn't compete for design time with the two units above.

3.7 Verification stack

A four-piece verification methodology - golden Python reference model, instruction generator, software instruction-level simulator, and a SystemVerilog top-level testbench doing a bit-exact comparison against DRAM output - applied to this transformer design from first principles:

Piece	Purpose
golden_transformer_ref.py	NumPy reference implementation of one transformer layer: QKV projection, scaled dot-product attention with causal masking, softmax, FFN, LayerNorm, with the exact quantization math the RTL will use.
instr_generator.py	Emits the instruction stream and DRAM stimulus image directly from the golden model's tensors and the ISA defined in 3.4.
instr_simulator.py	A software, instruction-level simulator that cross-checks the generator's output and produces per-instruction SRAM/DRAM checkpoints for RTL waveform debugging.
tb_top.sv	Full-chip SystemVerilog testbench with an AXI DRAM model; byte-compares RTL DRAM output against the golden reference output.
Unit testbenches	Independent testbenches per block (matmul PE array, attention/softmax unit, DMA, SRAM arbiter, dispatcher) run and passing before top-level integration.

4. Four-Week Execution Timeline

Week	Focus	Concrete deliverables
1	Reading + architecture spec + matmul engine RTL	Finish reading list sections 2.1-2.3. Write a 2-3 page architecture spec (block diagram, ISA table, precision plan, memory sizing math, all worked from first principles). RTL: systolic matmul engine plus a passing unit testbench.
2	Attention/softmax unit + memory system	RTL: attention unit ($QK^T \rightarrow \text{softmax} \rightarrow x \cdot V$) with its own unit testbench. RTL: SRAM banks, arbiters, double buffering. Finish reading list section 2.4.
3	DMA + dispatcher + full integration	RTL: Read/Write DMA, instruction dispatcher, KV-cache append logic. Wire everything into a top-level module. Begin the golden reference model and instruction generator in Python.
4	Verification, synthesis, writeup	Full instruction-level simulator, top-level testbench, bit-exact match on at least a small and a larger configuration. Run synthesis (open-source flow, or institute EDA tools if available) for area and power. Produce a roofline plot and a written report at paper-draft quality. Push a clean GitHub repo with its own README, its own module-ownership section (you, throughout), and its own Makefile-driven build/test flow.

If Week 4 verification is not fully bit-exact by the deadline, ship what is verified - for example, matmul path fully verified with the attention path partially verified and its gaps explicitly documented. An honest, partially verified real design is a stronger interview asset than a claimed-complete, unverified one.

5. Mapping To Etched's RTL Intern JD

JD line	How this project answers it
"help design microarchitecture and implement logic in verilog"	The entire architecture in Section 3 is independently designed and implemented in SystemVerilog by one person.
"work with cutting-edge machine learning architectures"	The hardware target is a transformer/attention layer, designed from the original papers, not repurposed CNN/SNN hardware.
"contribute to RTL block development"	Matmul engine, attention/softmax unit, LayerNorm unit, DMA, dispatcher, SRAM arbiters - multiple independent, unit-tested RTL blocks.
"full design cycle - microarchitecture discussions to synthesis and timing feedback"	Architecture spec $\rightarrow$ RTL $\rightarrow$ verification $\rightarrow$ synthesis $\rightarrow$ area/power/timing writeup covers the entire loop end to end.
"do not necessarily need prior ML/AI hardware experience... ability to learn quickly"	This project demonstrates that capability concretely with a finished artifact, rather than asserting it in an interview.
"familiarity with SystemVerilog, UVM, or Python" / writing testbenches	Four-piece verification stack (golden model, generator, simulator, SV testbench) applied here to a harder, transformer-native problem.
"familiarity with physical design flows and tooling"	Directly addressed by the synthesis run and area/power analysis section.
"familiarity with modern ML and LLM model architectures" / "numerical representations"	Directly addressed by the reading list (2.1, 2.2) and the from-scratch precision plan in Section 3.3.

6. Final Deliverables Checklist

- A GitHub repo with its own structure (rtl/, testbench/, sim/, docs/), its own README documenting the ISA and module ownership (all yours), and its own Makefile-driven build and simulation flow.
- An architecture spec document (2-3 pages), written before heavy RTL work and revised as the design evolves.
- Full RTL: matmul engine, attention/softmax unit, LayerNorm unit, SRAM subsystem, DMA (read and write), instruction dispatcher, top-level integration.
- Verification: golden Python reference model, instruction generator, software instruction-level simulator, top-level SystemVerilog testbench, per-block unit testbenches, at least one small and one larger configuration passing bit-exact.
- Synthesis results: area and power from a real EDA run - rough, honestly-reported numbers from a solo, intern-scale project are expected and fine; the point is running the flow, not hitting SOTA.
- A roofline plot and a short energy/latency discussion written at the same quantitative standard as a paper draft.
- A short 'what I'd do with more time or more people' section - shows engineering maturity and self-awareness about scope, and turns this from a resume line into an interview conversation.

Read every paper from its original source (arXiv / ACM DL / IEEE Xplore); this document does not reproduce their content. Prior accelerator work (IITM BTP report, COMPASS-CNN submission) is referenced here only as a bar for scope, correctness, and reporting rigor - all architecture, precision, and ISA decisions above are worked out independently for a transformer.